Build a YouTube Downloader

Single videos and whole playlists with Python and yt-dlp

A practical, step-by-step Python project

Start with an empty folder. Finish with a small program that saves one video, or a whole playlist in a folder with the playlist's name, using the exact YouTube titles.

Alireza Khajehvandi

Portfolio • GitHub • LinkedIn

YouTube • Telegram • Instagram

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

How to use this tutorial

This is a hands-on guide for people who are new to Python. Complete the chapters in order and stop at every *checkpoint*. Commands are ready to copy, but read the short explanation beside each one so you know what it does.

- Chapters 1–2 explain the goal and prepare your computer.
- Chapters 3–4 teach every Python tool and every `yt-dlp` setting the program uses, each with a tiny example you can try.
- Chapter 5 builds `downloader.py` step by step until it is complete.
- Chapter 6 runs the program for videos, playlists, and audio, and tests it.
- Chapter 7 shows how real use revealed problems, how their causes were found, and why each fix works.
- Chapter 8 solves common problems and suggests practice ideas.

The English and Persian editions contain the same code, commands, expected results, and advice. Only their language and page direction differ. The complete project and both PDF editions are published on GitHub.

Important

Download only videos that you own, that are offered under a licence that allows it, or that you have permission to save. Respect YouTube's Terms of Service and the creators' copyrights. This tutorial teaches Python; how you use the program is your responsibility.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

Contents

1 What We Will Build **1**

1.1 The program's behaviour . . . 1
1.2 How it works . . . 2
1.3 Project structure . . . 2
1.4 What you need . . . 2

2 Prepare Python and the Project **4**

2.1 Check Python . . . 4
2.2 Install FFmpeg . . . 4
2.3 Get the project folder . . . 5
2.4 Create an isolated environment . . . 5
2.5 Write the requirements file . . . 6
2.6 Install the packages . . . 6

3 The Python Tools We Use **7**

3.1 Modules and `import` . . . 7
3.2 Variables, strings, and `in` . . . 7
3.3 Functions, `return`, and default values . . . 8
3.4 Decisions with `if` and `else` . . . 8
3.5 Lists, tuples, and dictionaries . . . 8
3.6 Paths with `pathlib` . . . 9
3.7 Finding a program with `shutil.which` . . . 9
3.8 The `with` statement . . . 9
3.9 Command-line arguments with `argparse` . . . 10
3.10 Asking the user with `input` . . . 10
3.11 Raising your own errors with `raise` . . . 11
3.12 Running another program with `subprocess.run` . . . 11
3.13 Handling errors with `try` and `except` . . . 11

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

3.14 Replacing text with `replace` . . . 12
3.15 The `if __name__ == "__main__"` line . . . 12

4 Meet yt-dlp **13**
4.1 A first look from the terminal . . . 13
4.2 Using yt-dlp from Python . . . 13
4.3 The settings our program uses . . . 14
4.3.1 Output templates: naming the files . . . 14
4.3.2 Format selection: choosing the quality . . . 14
4.3.3 Choosing a quality . . . 15
4.3.4 Preferring formats that play everywhere . . . 15
4.3.5 Only the sound: post-processors . . . 15
4.3.6 Cookies: when YouTube asks you to sign in . . . 16

5 Write the Program Step by Step **17**
5.1 Step 1: describe the file . . . 17
5.2 Step 2: import the tools and set the defaults . . . 17
5.3 Step 3: recognise a playlist . . . 18
5.4 Step 4: choose the file names . . . 19
5.5 Step 5: choose the quality . . . 19
5.6 Step 6: collect all settings . . . 20
5.7 Step 7: check that FFmpeg really works . . . 21
5.8 Step 8: download . . . 22
5.9 Step 9: read the command line . . . 23
5.10 Step 10: start the program . . . 24
5.11 The complete program . . . 24

6 Run and Test the Program **29**
6.1 Show the help page . . . 29
6.2 Download a single video . . . 30
6.3 Download a whole playlist . . . 31
6.4 Save only the sound . . . 31
6.5 Choose the video quality . . . 31
6.6 Use a specific FFmpeg . . . 32
6.7 Choose the save folder . . . 33
6.8 Other ways to run it . . . 33

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

6.9 Stop and continue later . . . 33
6.10 The offline tests . . . 33

7 Lessons from Real Use **37**
7.1 A simple method for any bug . . . 37
7.2 Lesson 1: a network drop left half a video . . . 37
7.2.1 The symptom . . . 37
7.2.2 Reading the evidence . . . 38
7.2.3 Finding the cause . . . 38
7.2.4 The fix and the test . . . 38
7.3 Lesson 2: the terminal changed the link . . . 39
7.3.1 The symptom . . . 39
7.3.2 Why it matters . . . 39
7.3.3 The fix . . . 39
7.4 Lesson 3: keep downloads out of the code and out of Git . . . 39
7.5 Lesson 4: a new feature—only the sound . . . 40
7.6 Lesson 5: stopping politely . . . 40
7.7 Lesson 6: choosing the quality, and the vertical video . . . 41
7.7.1 The need and the first version . . . 41
7.7.2 The symptom . . . 41
7.7.3 The fix . . . 41
7.8 Lesson 7: finding FFmpeg is not the same as FFmpeg working . . . 41
7.9 Lesson 8: a comment must tell the truth . . . 42
7.10 Summary of the changes . . . 43

8 Troubleshooting and Next Steps **44**
8.1 Common problems . . . 45
8.2 Keep the project clean in Git . . . 46
8.3 Ideas for practice . . . 46

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

Chapter 1

What We Will Build

Chapter goal

Understand what the finished program does, how it works in one picture, and which files the project contains.

We will write one short Python program, `downloader.py`. You give it a YouTube link in the terminal and it saves the video on your computer.

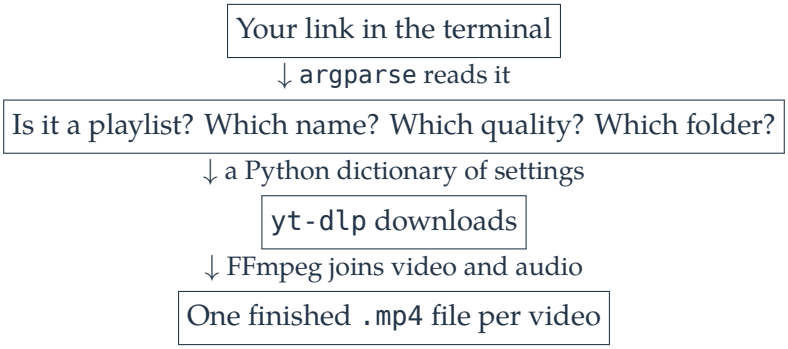
1.1 The program's behaviour

- **One video link:** the program downloads that video and names the file with its exact YouTube title, for example `Me at the zoo.mp4`.
- **A playlist link:** the program creates a folder with the playlist's title and downloads every video into it. Each file keeps its exact title and gets a number, so the files stay in the playlist order: `01 - First video.mp4`, `02 - Second video.mp4`, ...
- **A save folder:** with `-o FOLDER` you choose where the files go. Without it, files are saved in a `downloads` folder next to `downloader.py`, no matter where you start the program from. Git ignores this folder, so videos never end up in your repository.
- **Only the sound:** with `--audio` the program saves just the audio as an `.mp3` file—useful for music, podcasts, and language lessons.
- **Your choice of quality:** by default it saves 720p, or the nearest lower quality. `--quality 480` or `--quality 1080` chooses another one, and `--exact-quality` accepts only that exact quality. Video and audio are joined into one `.mp4` file that plays on phones, TVs, and every common player.
- **FFmpeg anywhere:** if FFmpeg is not found automatically, or the one found is broken, `--ffmpeg-location` points to a working copy. The program checks FFmpeg before downloading and explains clearly when it cannot run.
- **Friendly and robust:** if you forget the link, it asks for it. If one video in a playlist is private or deleted, it skips that video and continues. After a network drop it retries automatically, and `Ctrl+C` stops it politely so the same command can continue later.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

1.2 How it works

The hard work—talking to YouTube, choosing formats, downloading in parts, and showing progress—is done by `yt-dlp`, a popular open-source Python library. Our job is to *prepare the right settings* and hand the link to it.



1.3 Project structure

When you finish, your folder looks like this. Only the files in `code/` are needed to run the program; `docs/` holds this tutorial.

```
prj-youtube-downloader/
|-- code/
|   |-- downloader.py        # the complete program (218 lines)
|   |-- requirements.txt     # the packages to install
|   |-- test_downloader.py   # 18 offline tests
|   |-- README.md            # how to run the code
|   `-- downloads/           # your videos (created automatically, not in Git)
|-- docs/
|   |-- english/             # this book (English, XeLaTeX)
|   |-- persian/             # the same book in Persian (XePersian)
|   `-- README.md            # how to build the books
|-- .gitignore               # files Git must not save (videos, ...)
|-- LICENSE
`-- README.md                # project overview
```

1.4 What you need

- A computer with Windows, macOS, or Linux and an internet connection.
- Python 3.10 or newer (Chapter 2 shows how to check).
- FFmpeg, a free tool for joining video and audio (Chapter 2 installs it).
- A text editor. Visual Studio Code is a good free choice.

You do not need any previous experience with `yt-dlp`. Every Python idea used in the program is explained with an example in Chapter 3.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

Checkpoint

You can say in one sentence what happens for a single video link, what happens for a playlist link, and where files go when you do not choose a folder.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

Chapter 2

Prepare Python and the Project

Chapter goal

Check Python, install FFmpeg, create the project folder and an isolated Python environment, and install the packages.

2.1 Check Python

Open a terminal: *Terminal* on macOS and Linux, *PowerShell* on Windows, or the terminal inside your editor. Then run:

```
python --version
```

On some macOS or Linux systems the command is `python3`. Any result from `Python 3.10` upwards, such as `Python 3.12.11`, is suitable. If Python is missing, install it from `https://www.python.org/downloads/`. On Windows, tick *Add python.exe to PATH* in the installer.

2.2 Install FFmpeg

YouTube stores the best video and the best audio as two separate files. FFmpeg joins them into one `.mp4`. Install it with your system's package manager:

```
sudo apt install ffmpeg        # Ubuntu / Debian Linux
brew install ffmpeg            # macOS with Homebrew
winget install Gyan.FFmpeg     # Windows PowerShell
```

Close and reopen the terminal, then check it:

```
ffmpeg -version
```

The first line should begin with `ffmpeg version`. The program also works without FFmpeg, but then it can only save lower-quality files that already contain both video and audio (Section 4.3.2 explains why).

If you installed FFmpeg by unpacking a ZIP file (common on Windows) and did not add it to the `PATH`, the program cannot find it by itself. You do not need to change any system setting: give its location with `--ffmpeg-location`, either the program file or its folder (Chapter 6):

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
python downloader.py "LINK" --ffmpeg-location "C:\ffmpeg\bin"
```

2.3 Get the project folder

Choose *one* of the two ways below.

Way A: build everything yourself (recommended for learning)

Create an empty project folder with a `code` folder inside it, and enter it:

```
mkdir prj-youtube-downloader
cd prj-youtube-downloader
mkdir code
cd code
```

You will type every file in this folder while reading Chapters 2, 5, and 6.

Way B: download the finished project

```
git clone https://github.com/alirezaaies/prj-youtube-downloader.git
cd prj-youtube-downloader/code
```

If you downloaded a ZIP file from GitHub instead, extract it and open a terminal in its `code` folder.

2.4 Create an isolated environment

A *virtual environment* is a private box of packages for one project. It keeps this project's packages separate from other Python projects, so they never conflict. Use *one* of the following.

Python's built-in `venv` on macOS or Linux:

```
python3 -m venv .venv
source .venv/bin/activate
```

Python's built-in `venv` on Windows PowerShell:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

Conda (Anaconda or Miniconda), on any system:

```
conda create -n ytdl python=3.12 -y
conda activate ytdl
```

The terminal now shows `(.venv)` or `(ytdl)` before its prompt. Activate the environment again every time you open a new terminal for this project.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

Important

If PowerShell refuses to run `Activate.ps1`, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, answer `Y`, and try again.

2.5 Write the requirements file

A `requirements.txt` file lists the packages a project needs, so anyone can install all of them with one command. In Way A, create `requirements.txt` in the `code` folder with this content (Way B already has it):

Listing 2.1: code/requirements.txt

```
# yt-dlp talks to YouTube; [default] adds its recommended helpers.
# YouTube changes often, so keep yt-dlp up to date (pip install -U ...).
yt-dlp[default]>=2026.8.19
# JavaScript runtime that yt-dlp needs to read YouTube's player.
deno>=2.0
```

- Lines starting with # are comments for humans; pip ignores them.
- `yt-dlp[default]` installs `yt-dlp` plus its recommended helpers. The part in brackets is called an *extra*.
- `>=2026.8.19` means "this version or any newer one". We allow newer versions on purpose: YouTube changes often and `yt-dlp` is updated to follow it.
- `deno` is a small JavaScript engine. YouTube's web player contains JavaScript puzzles, and `yt-dlp` uses `deno` to solve them. Without it, some formats are missing or downloads fail.

2.6 Install the packages

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

The first line updates pip, Python's package installer. The second reads `requirements.txt` (`-r`) and installs everything listed in it. Writing `python -m pip` instead of only `pip` guarantees that the pip of the *active* environment is used.

Notice a small naming difference: the package is installed as `yt-dlp` (with a hyphen) but imported in Python as `yt_dlp` (with an underscore), because Python names cannot contain hyphens.

Checkpoint

Run the three commands below. Each prints a version number. If all three work, your environment is ready.

```
python -c "import yt_dlp; print(yt_dlp.version.__version__)"
deno --version
ffmpeg -version
```


Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

Chapter 3

The Python Tools We Use

Chapter goal

Learn every Python idea that appears in `downloader.py`, each with a tiny example you can run yourself.

You can try every example in the *Python interactive shell*. Type `python` in the activated terminal; the prompt changes to `>>>`. Type a line and press Enter to see its result. Leave the shell with `exit()`. Lines that begin with `#` in the examples are comments, and the expected output is shown in a comment such as `# -> True`.

3.1 Modules and `import`

A *module* is a file of ready-made Python code. `import` makes it available in your program. Some modules come with Python (the *standard library*); others, such as `yt_dlp`, are installed with pip.

```
import shutil                   # use it as shutil.which(...)
from pathlib import Path        # take only Path from the pathlib module
from yt_dlp import YoutubeDL    # take only YoutubeDL from yt_dlp
```

Our program uses three standard modules—`argparse` (read the command line), `shutil` (find programs), and `pathlib` (work with folders)—and one installed library, `yt_dlp`.

3.2 Variables, strings, and `in`

A *variable* is a name for a value. Text values are *strings* and are written in quotes. The `in` operator checks whether one string appears inside another and gives `True` or `False` (a *boolean*).

```
url = "https://www.youtube.com/playlist?list=PL123"
print("list=" in url)        # -> True
print("list=" in "watch?v=abc")  # -> False
```

An *f-string* (a string with `f` before the quote) puts values inside text with curly braces:

```
folder = "/home/sara/Videos"
print(f"Saving to: {folder}")   # -> Saving to: /home/sara/Videos
```


Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

The method `.strip()` removes spaces and line breaks from both ends of a string. This is useful for text that a user pastes:

```
print("  https://youtu.be/abc  ".strip())   # -> https://youtu.be/abc
```

3.3 Functions, `return`, and default values

A *function* is a named block of code that you can call again and again. `def` creates it; `return` sends a result back to the caller. The string right below the `def` line is a *docstring*: a short description for humans. Indented lines (four spaces) belong to the function.

```
def greet(name, greeting="Hello"):
    """Build a short greeting."""
    return f"{greeting}, {name}!"

print(greet("Sara"))          # -> Hello, Sara!
print(greet("Sara", "Hi"))    # -> Hi, Sara!
```

`greeting="Hello"` is a *default value*: it is used when the caller does not give that argument. Our `download` function uses the same idea for its folder and browser. The special value `None` means "nothing was given".

3.4 Decisions with `if` and `else`

`if` runs its block only when a condition is true; `else` runs otherwise. A `return` ends the function immediately, so code after it does not run.

```
def size_word(count):
    if count > 1:
        return "many videos"
    return "one video"

print(size_word(5))   # -> many videos
print(size_word(1))   # -> one video
```

Python also treats empty values as false: `""`, `None`, and `0` are false; a non-empty string is true. So `if browser:` means "if a browser name was given". The `or` operator returns its first true value:

```
print(None or "typed by user")      # -> typed by user
print("from command" or "typed")    # -> from command
```

3.5 Lists, tuples, and dictionaries

- A *list* holds values in order and can change: `["a", "b"]`.
- A *tuple* is like a list but cannot change: `("firefox",)`. A tuple with one value needs the trailing comma; `("firefox")` is just a string.
- A *dictionary* stores *key: value* pairs, like a small form with labelled fields.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
options = {"format": "best", "ignoreerrors": True}
print(options["format"])           # -> best
options["cookiesfrombrowser"] = ("firefox",)   # add a new key
print(len(options))                # -> 3
```

`yt-dlp` receives all of its settings as one dictionary, so this is the most important data type in our program.

3.6 Paths with `pathlib`

`Path` represents a file or folder path and works the same on Windows, macOS, and Linux.

```
from pathlib import Path

p = Path("~/Videos").expanduser()   # ~ becomes your home folder
print(p)                             # -> /home/sara/Videos
print(p / "Music")                   # -> /home/sara/Videos/Music
print(Path("code/downloader.py").resolve().parent)  # full folder path
p.mkdir(parents=True, exist_ok=True) # create it (and its parents) if missing
```

- `__file__` is a special variable that holds the path of the running Python file.
- `.resolve()` turns a path into a complete (absolute) path.
- `.parent` is the folder that contains the path.
- `exist_ok=True` means "no error if the folder already exists".

So `Path(__file__).resolve().parent` is "the folder that contains this script". That is exactly our default save folder.

3.7 Finding a program with `shutil.which`

`shutil.which(name)` looks for a command on your computer. It returns the command's path when it exists, and `None` when it does not:

```
import shutil
print(shutil.which("ffmpeg"))        # -> /usr/bin/ffmpeg
print(shutil.which("no-such-tool"))  # -> None
```

3.8 The `with` statement

Some objects must be opened and later closed properly. `with` does both: it opens the object, runs the indented block, and closes it at the end—even if an error happens.

```
with open("note.txt", "w") as f:   # open a file for writing
    f.write("Hello")
# here the file is already closed
```

We use `with YoutubeDL(...) as ydl:` in the same way.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

3.9 Command-line arguments with `argparse`

`argparse` reads the words typed after `python script.py`. It also creates a `--help` page and shows clear errors automatically. Save this as `hello.py`:

```
import argparse

parser = argparse.ArgumentParser(description="Say hello.")
parser.add_argument("name", nargs="?", help="who to greet")
parser.add_argument("-t", "--times", default="1", help="how many times")
args = parser.parse_args()
print(f"Hello {args.name}! " * int(args.times))
```

```
python hello.py Sara            # -> Hello Sara!
python hello.py Sara -t 2       # -> Hello Sara! Hello Sara!
python hello.py --help          # shows the automatic help page
```

- `"name"` has no dashes, so it is a *positional* argument: its value is given by position.
- `nargs="?"` makes it optional; when it is missing its value is `None`.
- `"-t", "--times"` is an *option*: a short and a long name for the same setting, with a default value.
- After `parse_args()`, values are available as `args.name` and `args.times`.

Some options are simple on/off switches that take no value. They are created with the setting `action="store_true"`. The value is then `True` when the option is written and `False` when it is not.

```
parser.add_argument("-l", "--loud", action="store_true", help="shout")
# python hello.py Sara          -> args.loud is False
# python hello.py Sara --loud   -> args.loud is True
```

3.10 Asking the user with `input`

`input(message)` shows a message, waits until the user presses Enter, and returns the typed text as a string:

```
link = input("Paste a link: ").strip()
print(f"You pasted {link}")
```

Everything typed in the terminal arrives as text. With `type=int`, `argparse` converts the value to a whole number, and it stops with a clear message when that is impossible:

```
parser.add_argument("--times", type=int, default=1)
# python hello.py Sara --times 3    -> args.times is the number 3
# python hello.py Sara --times abc  -> error: invalid int value: 'abc'
```


Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

3.11 Raising your own errors with `raise`

`raise` stops a function on purpose and reports an error. Choose the exception that describes the problem: `ValueError` for a wrong value, `RuntimeError` for something that went wrong while running.

```
def area(width, height):
    if width <= 0 or height <= 0:
        raise ValueError("sizes must be positive")
    return width * height
```

When you turn one error into another inside `except`, write `raise NewError(...) from exc`. The original error stays attached, which helps when you debug.

3.12 Running another program with `subprocess.run`

The `subprocess` module starts other programs, as if you typed them in the terminal. `subprocess.run` waits until the program ends and returns a result object.

```
import subprocess
result = subprocess.run(["ffmpeg", "-version"], capture_output=True,
                        text=True, timeout=15)
print(result.returncode)            # -> 0 means success
print(result.stdout.splitlines()[0])  # -> ffmpeg version 6.1.1 ...
```

- The command is a list: the program, then each argument.
- `capture_output=True` keeps the program's output in `result.stdout` and `result.stderr` instead of printing it. `text=True` returns it as strings instead of bytes.
- `timeout=15` raises `TimeoutExpired` if the program takes longer than 15 seconds.
- `returncode` is the program's *exit code*: 0 means success, any other number means failure. If the program cannot be started at all, Python raises an `OSError`, such as `FileNotFoundError`.

3.13 Handling errors with `try` and `except`

When something goes wrong while a program runs, Python raises an *exception* and, if nobody handles it, stops with a long error report (a *traceback*). `try` runs a block; if a named exception happens inside it, the matching `except` block runs instead of the crash.

```
try:
    number = int(input("Type a number: "))
except ValueError:
    print("That was not a number.")
```

Pressing `Ctrl+C` in the terminal raises the exception `KeyboardInterrupt`. Our program catches it to print one friendly sentence instead of a traceback. Catch only the exceptions you expect; catching every error would also hide real bugs.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

3.14 Replacing text with `replace`

`text.replace(old, new)` returns a copy of the string in which every `old` becomes `new`. Replacing with an empty string removes it. In Python code a single backslash is written `"\\"`, because one backslash alone starts an *escape sequence* such as `\n` (new line).

```
link = "https://youtu.be/abc\\?si\\=xyz"
print(link)                    # -> https://youtu.be/abc\?si\=xyz
print(link.replace("\\", ""))  # -> https://youtu.be/abc?si=xyz
```

3.15 The `if __name__ == "__main__"` line

When you run a file directly, Python sets its `__name__` variable to `"__main__"`. When another file imports it (for example our test file), `__name__` is the module's name instead. So this block means "start the program only when this file is run directly":

```
if __name__ == "__main__":
    main()
```

Thanks to it, the tests can import our functions without starting a download.

Try it yourself

Open the Python shell and run the `pathlib` and `shutil.which` examples on your own computer. Compare your results with the comments.

Checkpoint

You can explain what a function, a dictionary, `Path(__file__).resolve().parent`, `shutil.which`, `argparse`, `raise`, `subprocess.run`, and `try/except` are, in one sentence each.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

Chapter 4

Meet yt-dlp

Chapter goal

Understand how `yt-dlp` is used from Python and what each setting in our options dictionary does.

`yt-dlp` is a free, open-source downloader for YouTube and many other sites (`https://github.com/yt-dlp/yt-dlp`). It can be used in two ways: as a terminal command and as a Python library. Our program uses the library, but trying the command first helps you see what it does.

4.1 A first look from the terminal

```
yt-dlp --version
yt-dlp -F "https://www.youtube.com/watch?v=jNQXAC9IVRw"
```

The second command does not download anything. `-F` lists every *format* YouTube offers for that video. You will see audio-only rows, video-only rows, and a few rows with both. Notice that the best qualities are always *video only* or *audio only*. That is why we need FFmpeg to join them.

Important

Always put a YouTube link inside quotes in the terminal. Links often contain &, which the terminal would otherwise treat as a command separator.

4.2 Using yt-dlp from Python

From Python, `yt-dlp` needs only three steps: write a settings dictionary, create a `YoutubeDL` object with it, and call `download` with a *list* of links.

```
from yt_dlp import YoutubeDL

options = {"outtmpl": "%(title)s.%(ext)s"}
with YoutubeDL(options) as ydl:
    result = ydl.download(["https://www.youtube.com/watch?v=jNQXAC9IVRw"])
print(result)   # -> 0 means success
```


Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

`download` returns `0` when every link succeeded and another number when something failed. Our program uses this number to print its final message.

4.3 The settings our program uses

Key	What it does
`format`	Which video and audio to download.
`format_sort`	The order of preference, for example the quality first.
`ffmpeg_location`	Which FFmpeg to use, when it is not on the `PATH`.
`merge_output_format`	The container used when video and audio are joined: `mp4`.
`paths`	The base folder for downloads: `{"home": folder}`.
`outtmpl`	The *output template*: how each file is named.
`ignoreerrors`	`True`: skip a failing video and continue with the rest.
`retries`	Try again up to 10 times after a network drop.
`postprocessors`	Work done *after* a download, such as converting to MP3.
`cookiesfrombrowser`	Use the YouTube login of a browser: `("firefox",)`.

4.3.1 Output templates: naming the files

An output template is a normal string with special fields of the form `%(field)s`. `yt-dlp` replaces each field with information about the video:

Field	Replaced with
`%(title)s`	the exact video title
`%(ext)s`	the file extension, such as `mp4`
`%(playlist_title)s`	the playlist title
`%(playlist_index)02d`	the position in the playlist, as two digits

`s` means "as text" and `02d` means "as a whole number with at least two digits, padded with zero", so 3 becomes `03`. Padding keeps files sorted correctly in your file manager (`02` before `10`).

A `/` in the template creates a folder. So for a playlist named *Python Basics* whose third video is *Loops*, the template

```
%(playlist_title)s/%(playlist_index)02d - %(title)s.%(ext)s
```

produces `Python Basics/03 - Loops.mp4`. If a title contains characters that are not allowed in file names (such as `/` or `:` on Windows), `yt-dlp` replaces them safely by itself. Persian and other non-English titles are kept as they are.

4.3.2 Format selection: choosing the quality

The `format` value is a small expression:

```
bestvideo*+bestaudio/best
```


Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

- `bestvideo*` is the best video stream; the * also allows streams that happen to contain audio.
- `+bestaudio` adds the best audio stream; the two are joined by FFmpeg.
- `/best` is a fallback: "if that is impossible, take the best single file that already contains video and audio".

Joining needs FFmpeg. Without FFmpeg we ask only for `best`, which never needs joining. The best single files on YouTube are usually only 360p, so FFmpeg is strongly recommended.

4.3.3 Choosing a quality

The quality of a video is named after the *short side* of its picture. A wide 1280×720 video is "720p"; so is a vertical phone video of 720×1280. In `yt-dlp` this short side is called `res`. The program uses it in two ways.

Normal mode: a preference. The first rule of `format_sort` is `res:720`. It means: prefer the largest video *up to* 720; if there is none, take the smallest one above it. A preference never fails, so every video is downloaded, even an old one that only exists in 240p.

Exact mode: a filter. Square brackets after a format name keep only the formats that pass a test:

```
bestvideo*[height=1080][width>=1080]     wide video, exactly 1080p
bestvideo*[width=1080][height>=1080]     vertical video, exactly 1080p
```

A filter *can* fail. If no format passes, `yt-dlp` reports `Requested format is not available` and that video is skipped. Chapter 7 explains why a simple `[height<=720]` filter was not enough.

4.3.4 Preferring formats that play everywhere

`format_sort` also decides which of several good formats wins. YouTube often offers modern codecs (AV1, VP9, Opus) that some TVs, phones, and older players cannot open. The list

```
["res:720", "vcodec:h264", "acodec:m4a"]
```

means: first the requested quality, then H.264 video, then AAC (`m4a`) audio. The result is an `.mp4` file that plays everywhere, in the quality you chose.

4.3.5 Only the sound: post-processors

For audio only, the format is simply `bestaudio/best`: the best audio stream, or the best file if no separate audio exists. YouTube's best audio is an `.m4a` (AAC) or `.webm` (Opus) file. To get an `.mp3`, which every phone, car, and old player understands, we ask for a *post-processor*: a step that `yt-dlp` runs after the download.

```
"postprocessors": [
    {"key": "FFmpegExtractAudio", "preferredcodec": "mp3",
     "preferredquality": "192"},
]
```


Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

- `postprocessors` is a *list* because several steps can run one after another; each step is a dictionary.
- `"key"` names the step. `FFmpegExtractAudio` asks FFmpeg to convert the downloaded file to audio.
- `preferredcodec` chooses the output format and `preferredquality` the bit rate in kbit/s: `192` is clear speech and good music at a moderate size (about 1.4 MB per minute).

After the conversion `yt-dlp` deletes the original `.m4a`, so only the `.mp3` is left. The output template stays the same, so an audio file gets the same exact title, and a playlist gets the same numbered folder.

4.3.6 Cookies: when YouTube asks you to sign in

Sometimes YouTube answers with *Sign in to confirm you're not a bot*. This happens more often with VPNs and shared networks. A *cookie* is a small piece of data that a website stores in your browser to remember your login. With `cookiesfrombrowser`, `yt-dlp` reads the YouTube cookies of a browser on your own computer and uses your login, exactly as the browser would.

Important

Cookies are as private as a password. The program only reads them on your own computer; it never saves or sends them anywhere else. Never share cookie files and never put them in Git.

Checkpoint

Without running anything, write down the file name that the template `%(playlist_index)02d - %(title)s.%(ext)s` produces for the 7th video, titled *Functions*, saved as `mp4`. Answer: `07 - Functions.mp4`.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

Chapter 5

Write the Program Step by Step

Chapter goal

Create `downloader.py` piece by piece and understand every line. At the end of this chapter the program is complete.

Create an empty file named `downloader.py` in the `code` folder and type each piece below into it, in order. The line numbers in the margin are the line numbers of the finished file, so you can always see where a piece belongs. Leave two empty lines between functions; this is the usual Python style (PEP 8). The comments in the code (lines starting with `#`) explain each line as well, so the file stays understandable even without this book.

5.1 Step 1: describe the file

```
"""Download one YouTube video or a whole playlist with yt-dlp.

Examples:
    python downloader.py "VIDEO_OR_PLAYLIST_LINK"
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" -o ~/Videos
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --audio
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --quality 480
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --quality 1080 --exact-quality
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --ffmpeg-location /opt/ffmpeg/
    bin
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --browser firefox
    python downloader.py          # asks for the link
"""
```

A string at the very top of a file is the *module docstring*. Python ignores it while running, but it tells every reader what the file does and how to use it. Each example shows one option of the finished program.

5.2 Step 2: import the tools and set the defaults

```
import argparse
import shutil
import subprocess
```


Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
from pathlib import Path

from yt_dlp import YoutubeDL

# When no folder is given, files go to a "downloads" folder next to this script.
SCRIPT_FOLDER = Path(__file__).resolve().parent
DOWNLOAD_FOLDER = SCRIPT_FOLDER / "downloads"
# Video quality like YouTube's "720p": 360, 480, 720, 1080, etc. It is the
# short side of the picture (the height of a wide video).
DEFAULT_QUALITY = 720
# False: this quality or lower (or the nearest one above if nothing lower
# exists). True: exactly this quality, otherwise the video is skipped.
DEFAULT_EXACT_QUALITY = False
```

- `argparse` will read the command line, `shutil` will look for FFmpeg, `subprocess` will run FFmpeg once to check it, and `Path` will handle folders (Chapter 3).
- `YoutubeDL` is the downloader class from `yt_dlp` (Chapter 4).
- `SCRIPT_FOLDER` is the folder that contains `downloader.py`; `DOWNLOAD_FOLDER` is a `downloads` folder inside it. The `/` operator joins two parts of a path. The folder is created by the first download.
- `DEFAULT_QUALITY` is the video quality used when you do not choose one: 720, like YouTube's "720p". Quality means the *short side* of the picture: the height of a wide video, but the width of a vertical phone video (Chapter 4).
- `DEFAULT_EXACT_QUALITY` decides how strict that number is. With `False`, the program takes this quality or the nearest lower one.

Names in capital letters are a Python convention for values that never change (*constants*). Keeping them together at the top means you can change the program's defaults—for example to 1080—in one obvious place. Chapter 7 explains why the files go into their own `downloads` folder.

5.3 Step 3: recognise a playlist

```
def is_playlist(url):
    """Return True when the link contains a playlist id (list=...)."""
    return "list=" in url
```

Every YouTube playlist link contains `list=` followed by the playlist's id:

```
https://www.youtube.com/playlist?list=PLxxxxxxxx          -> playlist
https://www.youtube.com/watch?v=abc123&list=PLxxxxxxxx    -> playlist
https://www.youtube.com/watch?v=abc123                    -> one video
https://youtu.be/abc123                                   -> one video
```

The expression `"list=" in url` is already `True` or `False`, so we can return it directly. A link to a video that was opened *from* a playlist also contains `list=`; in that case the whole playlist is downloaded. To download only that single video, copy the link without the `&list=…` part.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

5.4 Step 4: choose the file names

```
37 def file_name_template(url):
38     """Tell yt-dlp how to name each downloaded file."""
39     if is_playlist(url):
40         # Playlist title/01 - Video title.mp4
41         return "%(playlist_title)s/%(playlist_index)02d - %(title)s.%(ext)s"
42     # Video title.mp4
43     return "%(title)s.%(ext)s"
```

This function returns one of the two output templates from Chapter 4. For a playlist, the `/` creates a folder with the playlist title and the number keeps the original order. For a single video, the file is simply named after its title. Notice the early `return`: if the link is a playlist, the function ends on line 41, so no `else` is needed.

5.5 Step 5: choose the quality

```
46 def video_quality(quality=DEFAULT_QUALITY, exact=DEFAULT_EXACT_QUALITY,
47                   ffmpeg_location=None):
48     """Build the yt-dlp format text for the requested quality.
49 
50     Quality means the short side of the picture, like YouTube's "720p": the
51     height of a wide video, but the width of a vertical (phone) video.
52     """
53     if quality <= 0:
54         raise ValueError("quality must be a positive number of pixels")
55     has_ffmpeg = ffmpeg_location or shutil.which("ffmpeg")
56     if not exact:
57         # No filter here: "res:<quality>" in format_sort (see build_options)
58         # picks the largest video up to this quality, or the smallest above it
59         # when nothing smaller exists, so a video is never skipped for quality.
60         return "bestvideo*+bestaudio/best" if has_ffmpeg else "best"
61     # Exact: a wide video must be exactly this high, a vertical one this wide.
62     wide = f"[height={quality}][width>={quality}]"
63     tall = f"[width={quality}][height>={quality}]"
64     if has_ffmpeg:
65         # Best video + best audio, joined by ffmpeg; after each "/" comes the
66         # next choice to try: a vertical video, then ready-made files.
67         return (f"bestvideo*{wide}+bestaudio/bestvideo*{tall}+bestaudio/"
68                 f"best{wide}/best{tall}")
69     # Without ffmpeg, only ready-made files that contain video and audio.
70     return f"best{wide}/best{tall}"
```

This function returns the `format` text for `yt-dlp` (Chapter 4).

1. Lines 53–54 refuse a quality of 0 or less with a `ValueError`, because such a number makes no sense. `raise` stops the function with that error (Chapter 3).

2. Line 55 remembers whether FFmpeg is available: either the location the user gave, or the one `shutil.which` finds. The value is a path (true) or `None` (false).

3. Lines 56–60, the normal mode: the format is only "best video + best audio" (or "best" without FFmpeg). The quality limit is *not* written here; it is the first rule of `format_sort` in the

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

next step. That rule always finds a video, even when the requested quality does not exist.

4. Lines 61–70, the exact mode: two filters describe "exactly this quality". `wide` is for normal videos, whose height is the short side; `tall` is for vertical videos, whose width is the short side. The format tries them one after another: `/` means "if that is impossible, try the next choice". If nothing matches, `yt-dlp` reports `Requested format is not available` and this video is skipped.

The long text on lines 67–68 is written as two f-strings inside parentheses. Python joins neighbouring strings automatically, which keeps each line short and readable.

5.6 Step 6: collect all settings

```
73  def build_options(url, folder, browser=None, audio=False, ffmpeg_location=None,
74                    quality=DEFAULT_QUALITY, exact_quality=DEFAULT_EXACT_QUALITY):
75      """Collect all settings in the dictionary that yt-dlp expects."""
76      options = {
77          "format": video_quality(quality, exact_quality, ffmpeg_location),
78          # The ffmpeg program yt-dlp must use (None lets yt-dlp search by itself).
79          "ffmpeg_location": ffmpeg_location or shutil.which("ffmpeg"),
80          # Order of preference: the requested quality first ("res" is the short
81          # side), then H.264 video and AAC (m4a) audio: an mp4 that plays
82          # everywhere.
83          "format_sort": [f"res:{quality}", "vcodec:h264", "acodec:m4a"],
84          "merge_output_format": "mp4",
85          "paths": {"home": str(folder)},
86          "outtmpl": file_name_template(url),
87          # Skip a video that fails (private, deleted, failed merge) and continue
88          # with the rest of the playlist instead of stopping.
89          "ignoreerrors": True,
90          # Retry after a network drop (the Python default is 0 retries).
91          "retries": 10,
92      }
93      if audio:
94          # Only the sound: the best audio stream; the video quality is ignored.
95          options["format"] = "bestaudio/best"
96          if options["ffmpeg_location"]:
97              # ffmpeg converts the downloaded audio to a 192 kbit/s mp3 file.
98              options["postprocessors"] = [
99                  {"key": "FFmpegExtractAudio", "preferredcodec": "mp3",
100                  "preferredquality": "192"},
101             ]
102     if browser:
103         # Reuse the YouTube login of this browser when YouTube asks for it.
104         options["cookiesfrombrowser"] = (browser,)
105     return options
```

This function builds the dictionary that `YoutubeDL` expects. Each key was explained in the tables in Chapter 4.

- `format` comes from the function we just wrote.
- `ffmpeg_location` tells `yt-dlp` which FFmpeg to use: the one the user gave, otherwise the one on the `PATH`. When it is `None`, `yt-dlp` searches by itself.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

- `format_sort` is the order of preference. Its first rule, `res:720` (with the chosen number), prefers the largest video up to that quality. Then H.264 video and AAC audio are preferred, so the resulting `.mp4` plays everywhere.
- `paths` receives the folder as a string (`str(folder)`), because that is what `yt-dlp` expects.
- `ignoreerrors` set to `True` means: when one video fails (private, deleted, a failed join, or no exact quality), report it, *skip it, and continue* with the rest of the playlist.
- `retries` is important. The `yt-dlp` *command* retries 10 times by default, but from *Python* the default is 0 (Chapter 7).
- Lines 93–101 run only for `--audio`. They *replace* the format with `bestaudio/best` (only the sound; the quality setting does not matter) and, when FFmpeg exists, add a *post-processor* that converts the result to MP3. Without FFmpeg the original audio file (usually `.m4a`) is kept.
- Lines 102–104 add the browser login only when `--browser` was given. `(browser,)` is a tuple with one item.

Building the dictionary first and then changing a few keys with `if` blocks is a common pattern: the normal case stays readable, and each special case is one small, separate block.

5.7 Step 7: check that FFmpeg really works

```
108 def check_ffmpeg(location):
109     """Run "ffmpeg -version" once: finding the file does not prove it works."""
110     if not location:
111         return  # No ffmpeg: the program still works with ready-made files.
112     executable = Path(location).expanduser()
113     if executable.is_dir():
114         # A folder was given: look for the ffmpeg program inside it
115         # (shutil.which also finds "ffmpeg.exe" on Windows).
116         executable = executable / "ffmpeg"
117         executable = Path(shutil.which(str(executable)) or str(executable))
118     try:
119         result = subprocess.run(
120             [str(executable), "-version"], capture_output=True, text=True,
121             errors="replace", timeout=15,
122         )
123     except (OSError, subprocess.TimeoutExpired) as exc:
124         # OSError: the file is missing or not executable. Timeout: it hangs.
125         raise RuntimeError(f"Cannot run FFmpeg at {executable}: {exc}") from exc
126     if result.returncode:
127         # The program started but reported an error (for example a broken build
128         # or a missing system library). Show its own message to the user.
129         details = result.stderr.strip() or result.stdout.strip()
130         raise RuntimeError(
131             f"FFmpeg exists but cannot run: {executable} "
132             f"(exit code 0x{result.returncode & 0xffffffff:08X}). "
133             "Repair the FFmpeg installation or use --ffmpeg-location with a "
134             f"working build. {details}"
135         )
```


Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

Finding a file called `ffmpeg` does not prove that it works: it can be a broken or half-installed copy that crashes when it starts. Then every download would fail at the very end, while joining video and audio. This function runs `ffmpeg -version` once, before anything is downloaded.

1. Lines 110–111: without FFmpeg there is nothing to check. The program still works with ready-made files, so `return` simply ends the function.
2. Lines 112–117: the user may give a folder instead of the program file. Then we look for `ffmpeg` inside it. `shutil.which` with a full path also finds `ffmpeg.exe` on Windows.
3. Lines 118–122: `subprocess.run` starts the program with the argument `-version`. `capture_output=True` keeps its text instead of printing it, and `timeout=15` stops waiting after 15 seconds (Chapter 3).
4. Lines 123–125: an `OSError` means the file is missing or cannot be started; `TimeoutExpired` means it hangs. Both become one clear `RuntimeError`. `from exc` keeps the original error attached for debugging.
5. Lines 126–135: the program started but returned a non-zero *exit code*, which means failure. The message contains the path, the exit code, advice, and FFmpeg's own error text. The exit code is printed in hexadecimal (`0x…`) because Windows reports such errors that way, for example `0xC0000135` for "a required DLL is missing".

5.8 Step 8: download

```
138 def download(url, folder=DOWNLOAD_FOLDER, browser=None, audio=False,
139              ffmpeg_location=None, quality=DEFAULT_QUALITY,
140              exact_quality=DEFAULT_EXACT_QUALITY):
141     """Download the link into folder. Return 0 when everything succeeded."""
142     # Use the ffmpeg the user gave, otherwise the one found on the PATH.
143     if ffmpeg_location:
144         location = str(Path(ffmpeg_location).expanduser())
145     else:
146         location = shutil.which("ffmpeg")
147     # Stop early with a clear message if that ffmpeg cannot run.
148     check_ffmpeg(location)
149 
150     folder = Path(folder).expanduser().resolve()
151     folder.mkdir(parents=True, exist_ok=True)
152     print(f"Saving to: {folder}")
153 
154     options = build_options(url, folder, browser, audio, location,
155                             quality, exact_quality)
156     with YoutubeDL(options) as ydl:
157         return ydl.download([url])
```

1. Lines 143–146 choose the FFmpeg to use: the location from `--ffmpeg-location` (with `~` expanded), otherwise the one found on the `PATH`.
2. Line 148 checks it. If it cannot run, `check_ffmpeg` raises an error and nothing is downloaded.
3. Line 150 turns the save folder into a complete path: `expanduser` replaces `~` with the home folder and `resolve` makes a relative path absolute. Line 151 creates it; line 152 prints it.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

4. Lines 154–157 build the settings, create the downloader, download the link, and return `yt-dlp`'s result: `0` when every video succeeded.

5.9 Step 9: read the command line

```
160 def main():
161     parser = argparse.ArgumentParser(
162         description="Download a YouTube video or a whole playlist."
163     )
164     parser.add_argument("url", nargs="?", help="video or playlist link")
165     parser.add_argument(
166         "-o", "--output", default=DOWNLOAD_FOLDER,
167         help="folder for the files (default: downloads/ next to this script)",
168     )
169     parser.add_argument(
170         "-a", "--audio", action="store_true",
171         help="save only the sound, as an mp3 file",
172     )
173     parser.add_argument(
174         "--browser",
175         help="use this browser's YouTube login, e.g. firefox or chrome",
176     )
177     parser.add_argument(
178         "--ffmpeg-location",
179         help="path to a working ffmpeg executable or its containing folder",
180     )
181     parser.add_argument(
182         "--quality", type=int, default=DEFAULT_QUALITY,
183         help=f"video quality such as 480 or 720: this or lower "
184              f"(default: {DEFAULT_QUALITY})",
185     )
186     parser.add_argument(
187         "--exact-quality", action="store_true", default=DEFAULT_EXACT_QUALITY,
188         help="require exactly this quality; skip the video if unavailable "
189              "(ignored for audio)",
190     )
191     args = parser.parse_args()
192     # type=int already rejects text; this also rejects 0 and negative numbers.
193     if args.quality <= 0:
194         parser.error("--quality must be a positive number, such as 720")
195 
196     url = args.url or input("Paste a YouTube video or playlist link: ").strip()
197     url = url.replace("\\", "")  # zsh may paste ? and = as \? and \=
198     if not url:
199         parser.error("a link is required")
200 
201     try:
202         result = download(url, args.output, args.browser, args.audio,
203                           args.ffmpeg_location, args.quality, args.exact_quality)
204     except RuntimeError as exc:
205         # A broken ffmpeg: print one clear line and exit with code 1.
206         parser.exit(1, f"Error: {exc}\n")
207     except KeyboardInterrupt:
208         print("\nStopped. Run the same command again to continue.")
209         return
```


Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
210
211     if result == 0:
212         print("Done. All files were downloaded.")
213     else:
214         print("Finished, but some files could not be downloaded (see the errors
        above).")
```

`main` connects the user to the functions above.

- `url` is positional and optional (`nargs="?"`).
- `-o` chooses the folder, `-a` switches to audio only (`action="store_true"`), and `--browser` names a browser for cookies.
- `--ffmpeg-location` (line 178) accepts the FFmpeg program or its folder, for when FFmpeg is not on the `PATH` or the one there is broken.
- `--quality` (line 182) uses `type=int`: `argparse` converts the text `"480"` to the number 480 and rejects text such as `abc` with a clear message.
- `--exact-quality` (line 187) is an on/off switch.
- Lines 193–194 also reject 0 and negative numbers, which `type=int` accepts.
- Line 196 asks for the link if none was given; line 197 removes backslashes that zsh may add when you paste a link (Chapter 7); lines 198–199 stop on an empty link.
- Lines 201–209 run the download inside `try`. A `RuntimeError` (a broken FFmpeg) is printed as one line, and `parser.exit(1, …)` ends the program with exit code 1, which tells scripts that it failed. `Ctrl+C` (`KeyboardInterrupt`) prints one friendly sentence.
- Lines 211–214 print a final message based on the result.

5.10 Step 10: start the program

```
217 if __name__ == "__main__":
218     main()
```

These two lines call `main` only when the file is run directly, not when the test file imports it (Chapter 3).

5.11 The complete program

Your file should now be identical to the listing below. Compare them line by line if anything does not work.

Listing 5.1: The complete code in code/downloader.py

```
1  """Download one YouTube video or a whole playlist with yt-dlp.
2
3  Examples:
4      python downloader.py "VIDEO_OR_PLAYLIST_LINK"
5      python downloader.py "VIDEO_OR_PLAYLIST_LINK" -o ~/Videos
```


Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --audio
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --quality 480
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --quality 1080 --exact-quality
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --ffmpeg-location /opt/ffmpeg/
    bin
    python downloader.py "VIDEO_OR_PLAYLIST_LINK" --browser firefox
    python downloader.py          # asks for the link
"""

import argparse
import shutil
import subprocess
from pathlib import Path

from yt_dlp import YoutubeDL

# When no folder is given, files go to a "downloads" folder next to this script.
SCRIPT_FOLDER = Path(__file__).resolve().parent
DOWNLOAD_FOLDER = SCRIPT_FOLDER / "downloads"
# Video quality like YouTube's "720p": 360, 480, 720, 1080, etc. It is the
# short side of the picture (the height of a wide video).
DEFAULT_QUALITY = 720
# False: this quality or lower (or the nearest one above if nothing lower
# exists). True: exactly this quality, otherwise the video is skipped.
DEFAULT_EXACT_QUALITY = False


def is_playlist(url):
    """Return True when the link contains a playlist id (list=...)."""
    return "list=" in url


def file_name_template(url):
    """Tell yt-dlp how to name each downloaded file."""
    if is_playlist(url):
        # Playlist title/01 - Video title.mp4
        return "%(playlist_title)s/%(playlist_index)02d - %(title)s.%(ext)s"
    # Video title.mp4
    return "%(title)s.%(ext)s"


def video_quality(quality=DEFAULT_QUALITY, exact=DEFAULT_EXACT_QUALITY,
                  ffmpeg_location=None):
    """Build the yt-dlp format text for the requested quality.

    Quality means the short side of the picture, like YouTube's "720p": the
    height of a wide video, but the width of a vertical (phone) video.
    """
    if quality <= 0:
        raise ValueError("quality must be a positive number of pixels")
    has_ffmpeg = ffmpeg_location or shutil.which("ffmpeg")
    if not exact:
        # No filter here: "res:<quality>" in format_sort (see build_options)
        # picks the largest video up to this quality, or the smallest above it
        # when nothing smaller exists, so a video is never skipped for quality.
        return "bestvideo*+bestaudio/best" if has_ffmpeg else "best"
    # Exact: a wide video must be exactly this high, a vertical one this wide.
```

```python
    wide = f"[height={quality}][width>={quality}]"
    tall = f"[width={quality}][height>={quality}]"
    if has_ffmpeg:
        # Best video + best audio, joined by ffmpeg; after each "/" comes the
        # next choice to try: a vertical video, then ready-made files.
        return (f"bestvideo*{wide}+bestaudio/bestvideo*{tall}+bestaudio/"
                f"best{wide}/best{tall}")
    # Without ffmpeg, only ready-made files that contain video and audio.
    return f"best{wide}/best{tall}"


def build_options(url, folder, browser=None, audio=False, ffmpeg_location=None,
                  quality=DEFAULT_QUALITY, exact_quality=DEFAULT_EXACT_QUALITY):
    """Collect all settings in the dictionary that yt-dlp expects."""
    options = {
        "format": video_quality(quality, exact_quality, ffmpeg_location),
        # The ffmpeg program yt-dlp must use (None lets yt-dlp search by itself).
        "ffmpeg_location": ffmpeg_location or shutil.which("ffmpeg"),
        # Order of preference: the requested quality first ("res" is the short
        # side), then H.264 video and AAC (m4a) audio: an mp4 that plays
        # everywhere.
        "format_sort": [f"res:{quality}", "vcodec:h264", "acodec:m4a"],
        "merge_output_format": "mp4",
        "paths": {"home": str(folder)},
        "outtmpl": file_name_template(url),
        # Skip a video that fails (private, deleted, failed merge) and continue
        # with the rest of the playlist instead of stopping.
        "ignoreerrors": True,
        # Retry after a network drop (the Python default is 0 retries).
        "retries": 10,
    }
    if audio:
        # Only the sound: the best audio stream; the video quality is ignored.
        options["format"] = "bestaudio/best"
        if options["ffmpeg_location"]:
            # ffmpeg converts the downloaded audio to a 192 kbit/s mp3 file.
            options["postprocessors"] = [
                {"key": "FFmpegExtractAudio", "preferredcodec": "mp3",
                 "preferredquality": "192"},
            ]
    if browser:
        # Reuse the YouTube login of this browser when YouTube asks for it.
        options["cookiesfrombrowser"] = (browser,)
    return options


def check_ffmpeg(location):
    """Run "ffmpeg -version" once: finding the file does not prove it works."""
    if not location:
        return  # No ffmpeg: the program still works with ready-made files.
    executable = Path(location).expanduser()
    if executable.is_dir():
        # A folder was given: look for the ffmpeg program inside it
        # (shutil.which also finds "ffmpeg.exe" on Windows).
        executable = executable / "ffmpeg"
        executable = Path(shutil.which(str(executable)) or str(executable))
    try:
```

```
        result = subprocess.run(
            [str(executable), "-version"], capture_output=True, text=True,
            errors="replace", timeout=15,
        )
    except (OSError, subprocess.TimeoutExpired) as exc:
        # OSError: the file is missing or not executable. Timeout: it hangs.
        raise RuntimeError(f"Cannot run FFmpeg at {executable}: {exc}") from exc
    if result.returncode:
        # The program started but reported an error (for example a broken build
        # or a missing system library). Show its own message to the user.
        details = result.stderr.strip() or result.stdout.strip()
        raise RuntimeError(
            f"FFmpeg exists but cannot run: {executable} "
            f"(exit code 0x{result.returncode & 0xffffffff:08X}). "
            "Repair the FFmpeg installation or use --ffmpeg-location with a "
            f"working build. {details}"
        )


def download(url, folder=DOWNLOAD_FOLDER, browser=None, audio=False,
             ffmpeg_location=None, quality=DEFAULT_QUALITY,
             exact_quality=DEFAULT_EXACT_QUALITY):
    """Download the link into folder. Return 0 when everything succeeded."""
    # Use the ffmpeg the user gave, otherwise the one found on the PATH.
    if ffmpeg_location:
        location = str(Path(ffmpeg_location).expanduser())
    else:
        location = shutil.which("ffmpeg")
    # Stop early with a clear message if that ffmpeg cannot run.
    check_ffmpeg(location)

    folder = Path(folder).expanduser().resolve()
    folder.mkdir(parents=True, exist_ok=True)
    print(f"Saving to: {folder}")

    options = build_options(url, folder, browser, audio, location,
                            quality, exact_quality)
    with YoutubeDL(options) as ydl:
        return ydl.download([url])


def main():
    parser = argparse.ArgumentParser(
        description="Download a YouTube video or a whole playlist."
    )
    parser.add_argument("url", nargs="?", help="video or playlist link")
    parser.add_argument(
        "-o", "--output", default=DOWNLOAD_FOLDER,
        help="folder for the files (default: downloads/ next to this script)",
    )
    parser.add_argument(
        "-a", "--audio", action="store_true",
        help="save only the sound, as an mp3 file",
    )
    parser.add_argument(
        "--browser",
        help="use this browser's YouTube login, e.g. firefox or chrome",
```


Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
    )
    parser.add_argument(
        "--ffmpeg-location",
        help="path to a working ffmpeg executable or its containing folder",
    )
    parser.add_argument(
        "--quality", type=int, default=DEFAULT_QUALITY,
        help=f"video quality such as 480 or 720: this or lower "
             f"(default: {DEFAULT_QUALITY})",
    )
    parser.add_argument(
        "--exact-quality", action="store_true", default=DEFAULT_EXACT_QUALITY,
        help="require exactly this quality; skip the video if unavailable "
             "(ignored for audio)",
    )
    args = parser.parse_args()
    # type=int already rejects text; this also rejects 0 and negative numbers.
    if args.quality <= 0:
        parser.error("--quality must be a positive number, such as 720")

    url = args.url or input("Paste a YouTube video or playlist link: ").strip()
    url = url.replace("\\", "")  # zsh may paste ? and = as \? and \=
    if not url:
        parser.error("a link is required")

    try:
        result = download(url, args.output, args.browser, args.audio,
                          args.ffmpeg_location, args.quality, args.exact_quality)
    except RuntimeError as exc:
        # A broken ffmpeg: print one clear line and exit with code 1.
        parser.exit(1, f"Error: {exc}\n")
    except KeyboardInterrupt:
        print("\nStopped. Run the same command again to continue.")
        return

    if result == 0:
        print("Done. All files were downloaded.")
    else:
        print("Finished, but some files could not be downloaded (see the errors
    above).")


if __name__ == "__main__":
    main()
```

Checkpoint

Run `python downloader.py --help` in the `code` folder. You should see seven arguments: `url`, `-o`, `-a`, `--browser`, `--ffmpeg-location`, `--quality`, and `--exact-quality`. If Python shows a `SyntaxError` or an `IndentationError`, compare the line it names with the listing above.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

Chapter 6

Run and Test the Program

Chapter goal

Download a single video and a playlist, choose a save folder, and run the offline tests.

Make sure the environment is active (Chapter 2) and the terminal is in the `code` folder. On some systems you must type `python3` instead of `python`.

6.1 Show the help page

```
python downloader.py --help
```


Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

Expected result

```
usage: downloader.py [-h] [-o OUTPUT] [-a] [--browser BROWSER]
                     [--ffmpeg-location FFMPEG_LOCATION] [--quality QUALITY]
                     [--exact-quality]
                     [url]

Download a YouTube video or a whole playlist.

positional arguments:
  url                   video or playlist link

options:
  -h, --help            show this help message and exit
  -o OUTPUT, --output OUTPUT
                        folder for the files (default: downloads/ next to
    this
                        script)
  -a, --audio           save only the sound, as an mp3 file
  --browser BROWSER     use this browser's YouTube login, e.g. firefox or
                        chrome
  --ffmpeg-location FFMPEG_LOCATION
                        path to a working ffmpeg executable or its
    containing
                        folder
  --quality QUALITY     video quality such as 480 or 720: this or lower
                        (default: 720)
  --exact-quality       require exactly this quality; skip the video if
                        unavailable (ignored for audio)
```

6.2 Download a single video

We use the first video ever uploaded to YouTube because it is only 19 seconds long:

```
python downloader.py "https://www.youtube.com/watch?v=jNQXAC9IVRw"
```

`yt-dlp` prints what it is doing and a live progress line. The important lines look like this (paths are shortened):

Expected result

```
Saving to: /home/sara/prj-youtube-downloader/code/downloads
[youtube] Extracting URL: https://www.youtube.com/watch?v=jNQXAC9IVRw
[info] jNQXAC9IVRw: Downloading 1 format(s): 18
[download] Destination: .../code/downloads/Me at the zoo.mp4
[download] 100% of  ...
Done. All files were downloaded.
```

The file `Me at the zoo.mp4` is now in `code/downloads/`. This old video has no HD version, so one ready-made file (format `18`) was enough. For an HD video you will see two format numbers, such as `137+140`, and an extra line `[Merger] Merging formats into ...`: that is

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

FFmpeg joining the separate video and audio. Format numbers may differ on your computer; that is normal.

6.3 Download a whole playlist

Open any YouTube playlist, copy the link from the address bar, and run:

```
python downloader.py "https://www.youtube.com/playlist?list=PLAYLIST_ID"
```

Expected result

A new folder with the playlist's title appears in `code/downloads/`. The videos inside it are numbered in playlist order and keep their exact titles, for example:

```
Get more creative with AI/
|-- 01 - Vibe coding in Google AI Studio is changing the game.mp4
|-- 02 - 3 Incredible #NanoBanana Community Builds.mp4
`-- ...
```

A large playlist can take a long time. You can stop at any moment with `Ctrl+C`. If you run the same command again later, `yt-dlp` skips the videos that are already complete and continues with the rest.

6.4 Save only the sound

Add `-a` or `--audio`. It works for single videos and for playlists:

```
python downloader.py "https://youtu.be/huwi-cjPPXU" --audio
```

Expected result

```
[info] huwi-cjPPXU: Downloading 1 format(s): 140
[download] 100% of   14.00MiB in 00:00:04 at 2.98MiB/s
[ExtractAudio] Destination: .../code/downloads/Introduce Yourself
               in Slow German (for Absolute Beginners).mp3
Deleting original file ... .m4a (pass -k to keep)
Done. All files were downloaded.
```

Format `140` is YouTube's AAC audio. The `ExtractAudio` line is FFmpeg converting it to MP3; then the temporary `.m4a` is deleted. This 15-minute lesson becomes a 21 MB MP3 file. Only 14 MB had to be downloaded, instead of 144 MB for the full HD video.

6.5 Choose the video quality

Without `--quality` the program saves 720p, or the nearest lower quality. These real results come from a 53-second *vertical* video (a phone video, so the short side is its width):

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
python downloader.py "LINK"                                    # default: 720
python downloader.py "LINK" --quality 480
python downloader.py "LINK" --quality 1080 --exact-quality
python downloader.py "LINK" --quality 2160 --exact-quality
```

Expected result

```
[info] KXaSULPslqM: Downloading 1 format(s): 136+140-20      -> 720x1280
[info] KXaSULPslqM: Downloading 1 format(s): 135+140-20      -> 480x854
[info] KXaSULPslqM: Downloading 1 format(s): 137+140-20      -> 1080x1920
ERROR: [youtube] KXaSULPslqM: Requested format is not available. ...
Finished, but some files could not be downloaded (see the errors above).
```

The sizes on the right were measured with `ffprobe` after the downloads. The video has no 2160p (4K) version, so with `--exact-quality` it was skipped with a clear message instead of crashing. In a playlist, the other videos would still be downloaded. Without `--exact-quality`, a missing quality is never a problem: the program takes the nearest lower one, or the smallest higher one if nothing lower exists. `--quality` has no effect together with `--audio`.

6.6 Use a specific FFmpeg

The program normally finds FFmpeg by itself. When FFmpeg is installed somewhere else, give the program file or its folder with `--ffmpeg-location`:

```
python downloader.py "LINK" --ffmpeg-location /usr/bin                # Linux
    folder
python downloader.py "LINK" --ffmpeg-location "C:\ffmpeg\bin"         # Windows
    folder
python downloader.py "LINK" --ffmpeg-location ~/tools/ffmpeg          # program
    file
```

Before downloading anything, the program runs `ffmpeg -version` once. A wrong path or a broken FFmpeg stops it immediately with one clear line and exit code 1, instead of failing later on every video:

Expected result

```
Error: Cannot run FFmpeg at /no/such/ffmpeg: [Errno 2] No such file or
directory: '/no/such/ffmpeg'

Error: FFmpeg exists but cannot run: .../ffmpeg (exit code 0x0000007F).
Repair the FFmpeg installation or use --ffmpeg-location with a working
build. libfoo.so: cannot open shared object file
```

The second message comes from a deliberately broken test program that cannot load a library. The end of the message is FFmpeg's own explanation.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

6.7 Choose the save folder

Add `-o` and a folder. The folder is created if it does not exist:

```
python downloader.py "LINK" -o ~/Videos              # macOS / Linux
python downloader.py "LINK" -o "D:\My Videos"        # Windows
python downloader.py "LINK" -o downloads             # a folder here
```

The first line of the output, `Saving to: …`, always shows the complete path. Put the folder in quotes when its name contains spaces.

6.8 Other ways to run it

```
python downloader.py                            # asks for the link
python downloader.py "LINK" --browser firefox   # use Firefox's YouTube login
python downloader.py "LINK" --browser chrome -o ~/Videos
```

Without a link, the program shows `Paste a YouTube video or playlist link:`. Paste the link (usually `Ctrl+Shift+V` in a Linux terminal, right-click in Windows) and press Enter. Use `--browser` only when YouTube asks you to sign in (Chapter 8). Supported names include `firefox`, `chrome`, `chromium`, `edge`, `brave`, `opera`, and `safari`.

6.9 Stop and continue later

Press `Ctrl+C` at any moment. The program answers:

Expected result

```
Stopped. Run the same command again to continue.
```

The unfinished part stays in the folder as a `.part` file. When you run the same command again, `yt-dlp` prints `Resuming download at byte ...` and continues from there instead of starting over.

6.10 The offline tests

Tests are small programs that check other code automatically. Ours run in a fraction of a second and need no internet, because they check only how the program *prepares* a download. Create `test_downloader.py` next to `downloader.py`:

Listing 6.1: The complete code in code/test_downloader.py

```
"""Offline checks for downloader.py. Run with: python -m unittest -v"""

import subprocess
import unittest
from pathlib import Path
from unittest.mock import patch
```


Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```python

import downloader

VIDEO = "https://www.youtube.com/watch?v=abc123"
PLAYLIST = "https://www.youtube.com/playlist?list=PL123"


class DownloaderTests(unittest.TestCase):
    def test_detects_playlist_and_single_video(self):
        self.assertTrue(downloader.is_playlist(PLAYLIST))
        self.assertFalse(downloader.is_playlist(VIDEO))

    def test_single_video_is_named_by_title(self):
        self.assertEqual(downloader.file_name_template(VIDEO), "%(title)s.%(ext)s"
    )

    def test_playlist_gets_its_own_folder(self):
        name = downloader.file_name_template(PLAYLIST)
        self.assertTrue(name.startswith("%(playlist_title)s/"))

    def test_default_folder_is_downloads_next_to_script(self):
        script_folder = Path(downloader.__file__).resolve().parent
        self.assertEqual(downloader.DOWNLOAD_FOLDER, script_folder / "downloads")

    def test_options_use_given_folder_and_browser(self):
        options = downloader.build_options(VIDEO, "/tmp/videos", "firefox")
        self.assertEqual(options["paths"], {"home": "/tmp/videos"})
        self.assertEqual(options["cookiesfrombrowser"], ("firefox",))

    def test_retries_after_network_drop(self):
        self.assertEqual(downloader.build_options(VIDEO, "/tmp/videos")["retries"
    ], 10)

    def test_audio_only_becomes_mp3(self):
        with patch("shutil.which", return_value="/usr/bin/ffmpeg"):
            options = downloader.build_options(VIDEO, "/tmp/videos", audio=True)
        self.assertEqual(options["format"], "bestaudio/best")
        self.assertEqual(options["postprocessors"][0]["preferredcodec"], "mp3")

    def test_audio_only_without_ffmpeg_keeps_original_file(self):
        with patch("shutil.which", return_value=None):
            options = downloader.build_options(VIDEO, "/tmp/videos", audio=True)
        self.assertEqual(options["format"], "bestaudio/best")
        self.assertNotIn("postprocessors", options)

    def test_quality_without_ffmpeg(self):
        with patch("shutil.which", return_value=None):
            self.assertEqual(downloader.video_quality(), "best")

    def test_default_quality_prefers_720_or_lower(self):
        with patch("shutil.which", return_value="/usr/bin/ffmpeg"):
            options = downloader.build_options(VIDEO, "/tmp/videos")
        self.assertEqual(options["format"], "bestvideo*+bestaudio/best")
        self.assertEqual(options["format_sort"][0], "res:720")

    def test_quality_option_changes_the_preference(self):
        options = downloader.build_options(VIDEO, "/tmp/videos", quality=480)
```

```
        self.assertEqual(options["format_sort"][0], "res:480")

    def test_exact_quality_works_for_wide_and_vertical_videos(self):
        with patch("shutil.which", return_value="/usr/bin/ffmpeg"):
            text = downloader.video_quality(1080, exact=True)
        self.assertIn("bestvideo*[height=1080][width>=1080]+bestaudio", text)  # wide
        self.assertIn("bestvideo*[width=1080][height>=1080]+bestaudio", text)  # vertical

    def test_exact_quality_without_ffmpeg_uses_ready_made_files(self):
        with patch("shutil.which", return_value=None):
            text = downloader.video_quality(360, exact=True)
        self.assertEqual(text, "best[height=360][width>=360]/best[width=360][height>=360]")

    def test_quality_must_be_positive(self):
        with self.assertRaises(ValueError):
            downloader.video_quality(0)

    def test_given_ffmpeg_location_is_used(self):
        with patch("shutil.which", return_value=None):
            options = downloader.build_options(VIDEO, "/tmp/videos",
                                               ffmpeg_location="/opt/ffmpeg")
        self.assertEqual(options["ffmpeg_location"], "/opt/ffmpeg")
        self.assertTrue(options["format"].startswith("bestvideo*"))

    def test_no_ffmpeg_needs_no_check(self):
        self.assertIsNone(downloader.check_ffmpeg(None))

    def test_missing_ffmpeg_file_is_reported(self):
        with self.assertRaises(RuntimeError):
            downloader.check_ffmpeg("/no/such/folder/ffmpeg")

    def test_broken_ffmpeg_is_reported(self):
        broken = subprocess.CompletedProcess([], returncode=1, stdout="", stderr="bad build")
        with patch("subprocess.run", return_value=broken):
            with self.assertRaises(RuntimeError):
                downloader.check_ffmpeg("/usr/bin/ffmpeg")


if __name__ == "__main__":
    unittest.main()
```

- `unittest` is Python's built-in testing module. Each method whose name starts with `test_` inside a `unittest.TestCase` class is one test.
- `assertTrue`, `assertFalse`, and `assertEqual` check a result; if a check is wrong, the test fails and shows both values.
- `patch` temporarily replaces a function. In the last test, `shutil.which` pretends that FFmpeg is missing, so we can test that case even on a computer that has FFmpeg. In the same way, `test_broken_ffmpeg_is_reported` replaces `subprocess.run` with a fake result whose exit code is 1, so a broken FFmpeg can be tested without having one. The two audio tests

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

use the same trick in both directions: FFmpeg present (MP3 conversion is added) and FFmpeg missing (the original audio file is kept).

Run all tests:

```
python -m unittest -v
```

Expected result

```
test_audio_only_becomes_mp3 ... ok
test_audio_only_without_ffmpeg_keeps_original_file ... ok
test_broken_ffmpeg_is_reported ... ok
test_default_folder_is_downloads_next_to_script ... ok
test_default_quality_prefers_720_or_lower ... ok
test_detects_playlist_and_single_video ... ok
test_exact_quality_without_ffmpeg_uses_ready_made_files ... ok
test_exact_quality_works_for_wide_and_vertical_videos ... ok
test_given_ffmpeg_location_is_used ... ok
test_missing_ffmpeg_file_is_reported ... ok
test_no_ffmpeg_needs_no_check ... ok
test_options_use_given_folder_and_browser ... ok
test_playlist_gets_its_own_folder ... ok
test_quality_must_be_positive ... ok
test_quality_option_changes_the_preference ... ok
test_quality_without_ffmpeg ... ok
test_retries_after_network_drop ... ok
test_single_video_is_named_by_title ... ok
----------------------------------------------------------------------
Ran 18 tests in 0.001s

OK
```

Whenever you change `downloader.py`, run the tests again. If you change a behaviour on purpose, update the matching test too.

Checkpoint

You downloaded one video to `code/downloads`, saved one audio file with `--audio`, one playlist (or part of it) into its own folder, used `-o` and `--quality` once each, and saw `OK` from the tests.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

Chapter 7

Lessons from Real Use

Chapter goal

Follow how the first version of the program was improved after it was used on real videos: what went wrong, how the cause was found, why each fix works, and how it was tested. These are the habits that turn a working script into a reliable tool.

The first version of `downloader.py` passed all its tests and worked on short videos. Real use then showed problems that no test had covered. This chapter tells that story, because *finding* a problem teaches more than any finished code. Every change described here is already part of the program in Chapter 5.

7.1 A simple method for any bug

1. **Describe the symptom exactly.** What did you expect, and what happened?
2. **Read the whole output**, especially the first warning or error.
3. **Look at the evidence**: which files exist, how large they are, what their names say.
4. **Form a guess about the cause, then check it** in the documentation or the library's source code. Do not change code on a guess alone.
5. **Fix the cause, not the symptom**, with the smallest clear change.
6. **Add a test** that would have caught the problem, and repeat the real case.

7.2 Lesson 1: a network drop left half a video

7.2.1 The symptom

A learner downloaded a 15-minute, full HD lesson with this command:

```
python downloader.py "https://youtu.be/huwi-cjPPXU\?si\=Huhcwi9PkWR87c-u" -o ~/
    Videos
```

The output contained one error, and the program ended with the "some files could not be downloaded" message:

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
[info] huwi-cjPPXU: Downloading 1 format(s): 137+140
[download] Destination: ...Beginners).f137.mp4
[download] Got error: 9849475 bytes read, 318544 more expected:
[download] Destination: ...Beginners).f140.m4a
[download] 100% of   14.00MiB in 00:01:58 at 121.43KiB/s
Finished, but some files could not be downloaded (see the errors above).
```

The folder had an unfinished `.f137.mp4.part` video *without sound* and a `.f140.m4a` file *with only sound*. It looked as if the program had downloaded the video twice in two wrong formats.

7.2.2 Reading the evidence

- `137+140` means two formats: 137 is the 1080p video stream and 140 the AAC audio stream (Chapter 4). Downloading two files is *normal*; FFmpeg joins them at the very end. So the real question is: why was there no join?
- `Got error: … more expected` means the connection closed before the video part was complete. The speed (121 KiB/s) shows a slow, unstable network.
- There is no `Retrying (1/10)` line. The download was abandoned at the first error, and without a complete video part FFmpeg had nothing to join.

7.2.3 Finding the cause

The `yt-dlp` *command* is known to retry 10 times. Why not here? The answer is in the library's source code, which pip installed in the environment's `site-packages/yt_dlp` folder:

```
# options.py -- only used by the yt-dlp terminal command
'--retries', dest='retries', default=10, ...

# downloader/http.py -- used by everyone, including our program
for retry in RetryManager(self.params.get('retries'), self.report_retry):
```

The default of 10 belongs to the *command-line parser*. When we use `YoutubeDL` from Python, `params.get('retries')` finds nothing and returns `None`, which means *zero* retries.

Important

A library used from Python does not always behave like the same tool used from the terminal. Defaults often live in the command-line layer. When a setting matters, write it explicitly in your options instead of relying on a default.

7.2.4 The fix and the test

One line in `build_options` (Chapter 5, step 6):

```
"retries": 10,
```

A new test protects it, so nobody removes it by accident:

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
def test_retries_after_network_drop(self):
    self.assertEqual(downloader.build_options(VIDEO, "/tmp/videos")["retries"],
    10)
```

Then the real case was repeated. The same command continued the unfinished file (`Resuming download at byte 49650475`), downloaded the audio, and FFmpeg joined them. A check with `ffprobe` (installed together with FFmpeg) proved the result:

```
stream|codec_name=h264|codec_type=video|width=1920|height=1080|duration=906.64
stream|codec_name=aac|codec_type=audio|duration=906.69
```

Both streams are present and both last the full 15 minutes and 6 seconds.

7.3 Lesson 2: the terminal changed the link

7.3.1 The symptom

Look again at the link in the command above: `\?si\=`. The learner had pasted a normal link. The backslashes were added by *zsh*, the default shell on macOS and on many Linux systems. When you paste a URL, a zsh feature called *url-quote-magic* protects special characters such as `?` and `=` with a backslash, which would be correct *without* quotes. Inside quotes, however, the backslashes become part of the text, so Python received a broken link.

7.3.2 Why it matters

This video still worked by luck, because `yt-dlp` found the video id before the broken part. For a playlist link it would fail silently: `list\=` does not contain `list=`, so `is_playlist` returns `False` and the whole playlist would be saved without its folder and numbers.

7.3.3 The fix

A valid URL never contains a backslash, so it is always safe to remove them as soon as the link is read (Chapter 5, step 8):

```
url = url.replace("\\", "")  # zsh may paste ? and = as \? and \=
```

The lesson: *clean user input at the place where it enters the program*. Then every function after it can trust the value.

7.4 Lesson 3: keep downloads out of the code and out of Git

In the first version, files without `-o` were saved right next to `downloader.py`. After a few downloads the `code` folder was full of large videos, mixed with the source files, and one careless `git add .` could have tried to upload gigabytes to GitHub.

Now the default is a separate folder:

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

```
SCRIPT_FOLDER = Path(__file__).resolve().parent
DOWNLOAD_FOLDER = SCRIPT_FOLDER / "downloads"
```

and `.gitignore` contains the line `code/downloads/`. You can ask Git to confirm that a file is ignored and by which rule:

```
git check-ignore -v "code/downloads/Me at the zoo.mp4"
```

```
.gitignore:222:code/downloads/  code/downloads/Me at the zoo.mp4
```

Two independent protections now exist: the folder rule and the older rules for media extensions such as `*.mp4`. Separating *program* files from *data* files is a good habit in every project.

7.5 Lesson 4: a new feature—only the sound

Language learners, podcast listeners, and music lovers often need only the audio. Downloading a full HD video for that wastes time and space: the lesson above is 144 MB as video but needs only 14 MB of audio.

The design keeps the program small:

- One switch on the command line: `-a` / `--audio`, with `action="store_true"` (Chapter 3).
- One new parameter, `audio=False`, passed from `main` through `download` to `build_options`. Its default keeps the old behaviour, so nothing changes for existing commands.
- One `if audio:` block that changes only what is different. The format becomes `bestaudio/best`, and an FFmpeg post-processor converts the result to MP3 (Chapter 4).
- When FFmpeg is missing, the post-processor is *not* added, because it would fail. The original audio file is kept instead. This is the same "work with what the computer has" idea as in `video_quality`.

Two tests cover both cases. They use `patch` to pretend that FFmpeg exists or does not exist, so they give the same result on every computer.

7.6 Lesson 5: stopping politely

Pressing `Ctrl+C` during a download used to print a long traceback that looked like a crash. Now `main` catches `KeyboardInterrupt` (Chapter 3) and prints one clear sentence:

```
Stopped. Run the same command again to continue.
```

The sentence is true because of how `yt-dlp` works: an unfinished download is kept as a `.part` file, and the next run continues it. Good messages tell the user *what happened* and *what to do next*.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

7.7 Lesson 6: choosing the quality, and the vertical video

7.7.1 The need and the first version

Full HD files are large and slow on a weak connection, and 720p is often enough. So the program got a `--quality` option. The first version used a filter on the *height* of the picture:

```
bestvideo*[height<=720]+bestaudio/best[height<=720]
```

Its tests passed, because the tests only checked that this text was built correctly.

7.7.2 The symptom

The first real download used a short *vertical* video. With the default 720 the result was only 360×640, and `--quality 480` gave 240×426. Listing the formats with `yt-dlp -F` showed the reason:

```
18      mp4   360x640     30  2 |  ... avc1.42001E
136     mp4   720x1280    30    |  ... avc1.4d401f
137     mp4   1080x1920   30    |  ... avc1.640028
```

The 720p version (format 136) exists, but its *height* is 1280, so `[height<=720]` rejected it. YouTube names quality after the *short side* of the picture; for a vertical video that is the width. The exact mode had the same bug: `[height=1080]` selected a 608×1080 video, which is really "608p".

7.7.3 The fix

- Normal mode uses no filter. The limit became the first rule of `format_sort`: `res:720`. In `yt-dlp`, `res` *is* the short side, and a sort rule never rejects a video: it takes the largest one up to 720, or the smallest above if nothing smaller exists.
- Exact mode needs a filter, so it uses two: `[height=Q][width>=Q]` for wide videos and `[width=Q][height>=Q]` for vertical ones, tried one after the other.

After the fix the same video gave 720×1280, 480×854, and 1080×1920, and a wide video gave 1280×720. New tests check both orientations.

Important

A test that only checks that a text is built "as planned" cannot find a wrong plan. Always try a new feature on real, varied input: wide and vertical, old and new, short and long.

7.8 Lesson 7: finding FFmpeg is not the same as FFmpeg working

`shutil.which("ffmpeg")` only proves that a file with that name exists. On Windows in particular, a copied or half-installed FFmpeg can exist and still crash at start-up, for example with exit code `0xC0000135` ("a required DLL was not found"). Without a check, every single video would download completely and then fail at the last step, the join, wasting time and data.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

Two additions solve this:

- `--ffmpeg-location` lets the user point to a working FFmpeg, or to its folder, without changing any system setting.
- `check_ffmpeg` runs `ffmpeg -version` *once, before* the first download. A missing file, a crash, or a hang (15-second `timeout`) becomes one `RuntimeError` with the path, the exit code, advice, and FFmpeg's own message; `main` prints it as one line and exits with code 1.

This is the *fail fast* principle: check what you depend on at the start, and report it clearly, instead of failing later in a confusing place. The exit code is shown in hexadecimal because that is how Windows documents such errors; the expression `returncode & 0xffffffff` turns Python's negative number for those codes into the familiar unsigned form.

Three tests cover the check without needing a broken FFmpeg: no FFmpeg at all (nothing to check), a path that does not exist, and a fake `subprocess.run` result with exit code 1.

7.9 Lesson 8: a comment must tell the truth

During the review of these changes, one line read:

```
"ignoreerrors": True,  # Stop on errors, including a failed merge.
```

The code was right, but the comment said the opposite: `True` means *do not* stop; report the failed video, skip it, and continue with the rest of the playlist. A wrong comment is worse than no comment, because readers trust it. It now reads:

```
# Skip a video that fails (private, deleted, failed merge) and continue
# with the rest of the playlist instead of stopping.
"ignoreerrors": True,
```

This behaviour was then tested for real: a list that started with an unavailable video printed one `ERROR` line for it, still downloaded the next video, and ended with the "some files could not be downloaded" message. Whenever you change code, read the comments around it again.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

7.10 Summary of the changes

Problem or need	Change, and how it is tested
A network drop stopped the video	`"retries": 10`; a unit test and the real download repeated
zsh added backslashes to links	`url.replace(...)` in `main`; tested with a real zsh-escaped link
Videos mixed with the code	`DOWNLOAD_FOLDER` and `code/downloads/` in `.gitignore`; a unit test and `git check-ignore`
Only the sound	`--audio` and an MP3 post-processor; two unit tests and real single-video and playlist downloads
`Ctrl+C` looked like a crash	`try/except KeyboardInterrupt`; tested by stopping a real download
Choose the quality; vertical videos	`--quality`, `--exact-quality`, `res:` in `format_sort`; four unit tests and real wide and vertical downloads
FFmpeg not found or broken	`--ffmpeg-location` and `check_ffmpeg`; four unit tests and a real broken program
A wrong comment	the `ignoreerrors` comment corrected; a real list with an unavailable video

Checkpoint

You can explain why a full HD download creates two files before the final `.mp4`, why the Python default for `retries` differs from the terminal command, why the program removes backslashes from links, and why a height filter chooses the wrong quality for a vertical video.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

Chapter 8

Troubleshooting and Next Steps

Chapter goal

Fix the most common problems quickly and choose a small, useful improvement to build next.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

8.1 Common problems

Message or symptom	What to do
`ModuleNotFoundError: No module named 'yt_dlp'`	The environment is not active, or the packages are not installed. Activate it and run `python -m pip install -r requirements.txt`.
`Sign in to confirm you're not a bot`	Open YouTube in a browser and sign in. Close the browser, then run the command again with `--browser firefox` (or your browser's name). Turning a VPN off or changing its server can also help.
`No supported JavaScript runtime could be found`	`deno` is missing. Activate the environment or run `python -m pip install deno`.
`Got error: ... bytes read, ... more expected`, and the folder has a `.f137.mp4.part` file and a `.f140.m4a` file	The network dropped while the video part was downloading. YouTube sends video and audio as two parts, which FFmpeg joins only when both are complete. The program already retries 10 times; if it still fails, run the *same command* again. `yt-dlp` continues the unfinished part, joins both, and deletes the temporary files.
`Could not copy Chrome cookie database`	Chrome is open and locks its cookies. Close every Chrome window and try again, or use another browser.
The file has no sound, or only low quality is saved	FFmpeg is not installed or not found. Install it and check `ffmpeg -version` in a new terminal.
It worked before but now fails with `HTTP Error 403` or similar	YouTube changed something. Update the library: `python -m pip install -U "yt-dlp[default]"`.
`Video unavailable` or `Private video`	The video cannot be watched without permission. The other playlist videos are still downloaded and the program ends with the "some files could not be downloaded" message.
`Requested format is not available` after `--exact-quality`	That exact quality does not exist for this video, so it was skipped. Choose another `--quality`, or leave out `--exact-quality` to accept the nearest one.
`Error: Cannot run FFmpeg at ...`	The path given with `--ffmpeg-location` is wrong. Give the FFmpeg program itself or the folder that contains it (usually a `bin` folder).
`Error: FFmpeg exists but cannot run ... (exit code 0x...)`	That FFmpeg is broken or misses a library (`0xC0000135` on Windows means a missing DLL). Reinstall FFmpeg, or point `--ffmpeg-location` to a working copy. The end of the message is FFmpeg's own explanation.
`zsh: no matches found` or the link is cut at `&`	Put the link inside quotes.
A whole playlist starts when you wanted one video	The link contains `&list=`. Remove that part and run again.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

When you ask for help (for example in a GitHub issue), include your operating system, Python version, the exact command, and the complete error message. Remove any private information first.

8.2 Keep the project clean in Git

The project's `.gitignore` file tells Git which files never to save. It ignores the whole `code/downloads/` folder, media files anywhere (such as `*.mp4`, `*.mp3`, and `*.webm`), unfinished `*.part` files, and the `.venv` folder. Run `git status` before a commit: no video file should appear in the list.

8.3 Ideas for practice

Each idea needs only a few lines and practises one new skill. Add a test for each change.

1. **Choose the audio format.** Add an `--audio-format` option with `choices=["mp3", "m4a", "opus"]` and use its value as `preferredcodec` instead of the fixed `"mp3"`:

```
parser.add_argument("--audio-format", choices=["mp3", "m4a", "opus"],
                    default="mp3", help="file type for --audio")
```

2. **Show the available qualities.** Add a `--list` option that only prints the formats of a video, like `yt-dlp -F`, so the user can choose a `--quality` that exists:

```
options["listformats"] = True   # print the table, download nothing
```

3. **Several links at once.** Change `url` to `nargs="*"` and pass the whole list to `ydl.download`.

4. **Subtitles.** Save English and Persian subtitles next to each video:

```
options["writesubtitles"] = True
options["subtitleslangs"] = ["en", "fa"]
```

5. **Read links from a file.** Read one link per line from `links.txt`:

```
links = Path("links.txt").read_text().splitlines()
```

All available options are described in the `yt-dlp` README. An option written on the command line, such as `--write-subs`, becomes a dictionary key in Python, such as `writesubtitles`.

Checkpoint

You can find the cause of a failed download from its message, update `yt-dlp`, and add a new option to the settings dictionary yourself.

Congratulations—you have built a complete, tested Python tool from an empty folder.

Alireza Khajehvandi online
LinkedIn YouTube Telegram Instagram GitHub Portfolio

About the author

This tutorial and project were created by **Alireza Khajehvandi** to help new learners build something useful quickly and understand every line they run.

Continue learning and get in touch:

- LinkedIn
- YouTube channel
- Telegram channel
- Instagram
- GitHub
- Personal website and portfolio